Question 1: AI Drones for Emergency Medicine Delivery (Greedy Best-First Search and A*)

A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic: roads may suddenly become inaccessible and weather affects travel cost. The drone has a straight-line distance heuristic, partial real-time updates about blocked paths, and variable movement costs. It must decide routes quickly with minimum delay and risk.

Step 1: Understanding the Scenario

Search element	Representation in this scenario
State	Current location (waypoint or grid cell) of the drone, along with the known map
Actions	Fly to a neighbouring waypoint or road segment
Step cost c(n, n')	Distance × terrain/weather factor (for example, headwind, rain, flooded terrain increase cost)
Heuristic h(n)	Straight-line distance from n to the hospital or delivery point (G)
Goal	Deliver the medicine at G with minimum delay and risk
Uncertainty	Blocked paths appear suddenly; weather changes costs; updates are only partial

i) Behaviour of Greedy Best-First Search (GBFS)

GBFS expands the node that appears closest to the goal, using only the heuristic: $f(n) = h(n)$. The frontier is a priority queue ordered by the straight-line distance to G. The cost already spent, $g(n)$, is ignored.

Behaviour in the flood scenario:

1. The drone always heads to the neighbouring waypoint that is geographically nearest to the hospital.
2. It does not consider that this waypoint may lie over flooded terrain, strong winds, or a very expensive route segment.
3. If the nearest waypoint is blocked, the drone removes it and picks the next-nearest one. It may get trapped in a dead end (for example, a river between it and G) because straight-line closeness does not reflect reachability.

Strengths:

- Very fast decision making and low computation, which suits urgent situations.
- Low memory use because it expands few nodes when the heuristic is good.
- Easy to re-run when a path is blocked.

Weaknesses under uncertainty:

- Not optimal: it can choose a short-looking route with very high weather or terrain cost.

- Ignores variable costs and risk, so safety is not considered.
- Can be incomplete in graphs with loops or dead ends unless a closed list is used.
- Highly sensitive to heuristic errors. Straight-line distance is misleading when floods force detours.

ii) Behaviour of A* Search

A* ranks nodes using f(n) = g(n) + h(n), where g(n) is the actual accumulated cost from the start (including terrain and weather cost) and h(n) is the straight-line estimate to the goal. Therefore, A* balances what has been paid so far (g) against what is still expected (h).

How A* handles the situation:

- g(n) captures the real, variable cost of each segment, so costly stormy or flooded segments are penalized automatically.
- h(n) pulls the search toward the goal and avoids wandering in all directions like Uniform Cost Search.
- If h(n) never overestimates the true remaining cost (admissible), A* returns an optimal route. Here the straight-line distance is admissible as long as it is scaled by the minimum cost per unit distance (best-weather cost).
- When a path is blocked, the edge is deleted from the graph and the search continues with the other nodes in the open list or is re-run from the current position.

Worked comparison (illustrative values):

Candidate next waypoint	g(n)	h(n)	Greedy value f = h	A* value f = g + h
X (calm, longer straight-line distance)	2	4	4	6
Y (flooded, closer to G)	7	3	3	10

Greedy chooses Y (3 is lowest) even though the route through Y is expensive. A* chooses X (6 is lowest), which gives a cheaper and safer route overall.

iii) Impact of Heuristic Accuracy

Heuristic quality	Effect on Greedy Best-First Search	Effect on A*
Accurate (close to the true cost, admissible)	Moves directly to the goal and expands very few nodes, but still not guaranteed to be optimal	Expands very few nodes and returns an optimal path
Perfect h*(n)	Follows a good path but ignores costs if h is only distance-based	Expands only nodes on the optimal path
Underestimating (admissible but weak)	Behaves less directed and may explore more nodes	Still optimal, but expands more nodes (closer to Uniform Cost Search)
Zero (h = 0)	Degenerates to blind search	Becomes Uniform Cost Search: optimal but slow
Overestimating (inadmissible)	Can be strongly misled; may pick bad routes	Loses the optimality guarantee; may skip the cheapest path

Heuristic quality	Effect on Greedy Best-First Search	Effect on A*
Misleading (ignores floods, e.g. straight-line over a river)	Dead ends and large detours; most severe effect	Penalized by g(n), so the damage is smaller, but extra nodes are expanded

Conclusion: Greedy search depends entirely on the heuristic, whereas A* depends on it for efficiency only (not for correctness if h is admissible). Therefore A* is more robust when the heuristic is imperfect.

iv) Impact of Dynamic Changes (Blocked Paths)

- **Plan invalidation:** A blocked road invalidates the current plan. The drone must replan from its present position.
- **Heuristic remains valid:** Removing an edge can only raise the real cost, so the straight-line heuristic remains admissible. The search stays correct after updates.
- **Search performance:** GBFS replans cheaply but may repeatedly choose routes that get blocked again. A* replans more slowly (it expands more nodes), but it produces a lower-cost route each time.
- **Efficiency:** Frequent changes cause repeated search, higher computation, and possible delays. Incremental versions of A* (D* Lite, Lifelong Planning A*) reuse earlier search effort and are faster for such changes.
- **Cost changes:** Weather changes the edge costs. Costs in the open list must be updated, otherwise the drone follows stale information.

v) Recommended Algorithm and Justification

Recommendation: A* search with replanning (preferably the incremental variant, D* Lite, or Weighted A* when time is extremely short).

Criterion	Why A* with replanning is suitable
Real-time constraints	The heuristic focuses the search. Weighted A* ($f = g + w \cdot h$, w slightly above 1) or an anytime version can return a good route quickly, and D* Lite repairs the plan instead of restarting.
Optimality	With an admissible heuristic, A* gives the minimum-cost route for the known map. Greedy search gives no such guarantee.
Safety	Risk (flooded areas, strong wind) can be added to the edge cost g(n). The drone then avoids dangerous segments automatically.
Dynamic environment	Replanning from the current location after each update keeps the route valid when roads are blocked.

Greedy search may be used only as a fast fallback when the drone has an extremely short decision time and no better option, because it can lead the drone into dead ends or high-risk areas.

Question 2: Smart City Traffic Signal Optimization

A smart city AI system optimizes traffic signal timings across hundreds of intersections to minimize total waiting time, fuel consumption and traffic congestion. The system must continuously adjust signals based on traffic flow. The search space is extremely large, traffic patterns change dynamically, and the system may get stuck in locally optimal solutions.

Task: Formulation as an Optimization Problem and Why Traditional Search is Inefficient

Component	Formulation
State (solution)	x = (x1, x2, ..., xn), where xi = (cycle length, green split, offset) of intersection i
Objective function	Minimize J(x) = w1·W(x) + w2·F(x) + w3·C(x), where W = total waiting time, F = fuel consumption, C = congestion level. The weights w1, w2, w3 express priorities.
Constraints	Minimum and maximum green time for each phase; pedestrian crossing time; cycle length limits; coordination (offset) between neighbouring intersections; emergency vehicle priority
Neighbour move	Change the timing of one intersection slightly (for example, green time ±5 seconds, or shift an offset)
Evaluation	Traffic simulation or live sensor data gives J(x) for a candidate timing plan

Why traditional search is inefficient:

- **Huge search space:** If each of 100 intersections has 10 possible timing settings, there are 10^100 combinations. BFS, DFS, UCS and A* cannot enumerate or store this space.
- **No explicit goal state or path:** The problem is not about finding a path to a goal. Any complete plan is a state, and we only need the best state. The path to it is irrelevant.
- **Dynamic environment:** Traffic changes every few minutes, so the solution found by a long search quickly becomes outdated.
- **Memory and heuristic:** A good heuristic for exact search is hard to define, and tree-based methods store too many nodes.

Therefore, local search and metaheuristic methods, which keep only the current solution (or a small population) and improve it iteratively, are more appropriate.

i) Working of Hill Climbing and its Limitations

Hill Climbing steps:

1. Start with an initial timing plan (current signal settings).
2. Evaluate the cost J(x).
3. Generate neighbours by changing one intersection's timing.
4. Move to the best neighbour only if its cost is lower than the current cost.
5. Repeat until no neighbour improves the cost, then stop and return the current plan.

Illustration of getting stuck (cost values to be minimized):

Iteration	Current plan cost J	Best neighbour cost	Decision
0	120	110	Move (improvement)
1	110	105	Move
2	105	105 (all neighbours ≥ 105)	Stop: local optimum
Global optimum	80 (exists elsewhere in the space)	—	Never reached

Limitations:

- Gets stuck at local optima, plateaus and ridges (explained below).
- Result depends on the starting plan.
- Only accepts improvements, so it cannot escape bad regions.
- Hill climbing is fast and simple, but gives no guarantee of the global optimum.

ii) Local Maxima, Plateaus and Ridges: Real-World Interpretation

Problem	Meaning in search	Real-world traffic interpretation
Local optimum (local maximum in a maximization view, local minimum for cost)	A state better than all its neighbours but worse than the best state	A timing plan that clears the main road well, and any small change to one signal increases waiting time. However, a completely different coordination pattern across the city would reduce waiting much more.
Plateau	A flat region where neighbours have equal objective values	Changing a green time by a few seconds at one junction does not change total waiting time, because traffic is dominated by another bottleneck. The search has no direction to move.
Ridge	A narrow path of improvement that single-variable moves cannot follow	Improving requires changing two or more signals together (for example, shifting the offsets of consecutive junctions to form a 'green wave'). Changing one junction alone makes things worse, so hill climbing rejects every move.

iii) How Simulated Annealing and Genetic Algorithms Overcome These Limitations

Simulated Annealing (SA):

- SA picks a random neighbour. If it improves J, it is accepted. If it is worse by ΔE, it is accepted with probability e^(−ΔE/T), where T is the temperature.
- At high T, SA accepts many bad moves and explores widely (escaping local optima and plateaus). As T is slowly reduced (cooling schedule), it behaves like hill climbing and settles into a good solution.
- Real-world meaning: the system is allowed to temporarily make some intersections worse in order to find a better overall coordination.

Genetic Algorithms (GA):

- Encoding: a chromosome is the list of timing parameters of all intersections.
- Fitness: higher for lower J(x) (less waiting, fuel, congestion).
- Selection chooses good plans; crossover combines timings from two good plans (for example, the best plan for the north zone and the best for the south zone); mutation randomly changes some timings to keep diversity.
- A population explores many regions in parallel, so it is less likely to be trapped in one local optimum, and it can handle ridges because crossover changes several variables together.

iv) Recommended Approach and Justification

Recommendation: A Genetic Algorithm combined with local refinement (a memetic approach: GA for global exploration, hill climbing or simulated annealing for fine tuning), restarted from the previous best population when traffic changes.

Criterion	Justification
Scalability	GA works on a population of full solutions and its evaluations are independent, so they can be run in parallel across many processors. It handles hundreds of intersections without enumerating the space.
Adaptability	When traffic patterns change, the old population is reused as the starting point (warm start), so the search adapts quickly instead of starting from scratch.
Avoiding local optima	Population diversity, crossover and mutation avoid premature convergence. The local refinement step improves each candidate quickly.
Multiple objectives	The fitness function can combine waiting time, fuel and congestion, or use multi-objective GA (Pareto front) to trade them off.
Comparison	Hill climbing is fast but gets stuck. SA is simple and effective but works on a single solution and its slow cooling is hard to run in real time for a changing city. GA with local search gives the best balance.

Question 3: Autonomous Mars Rover Exploration

An autonomous rover on Mars explores unknown terrain. It must navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing, and make decisions without a complete map. It updates its knowledge as it explores, making this a real-time decision-making problem.

i) Why an Online Search Agent is Needed Instead of Offline Search

An offline search agent computes a complete plan before acting, which requires a known map. Online search interleaves computation and action: the agent acts, observes, and then decides the next step. The rover needs online search because:

- **Unknown environment:** The map is unknown. Offline algorithms such as A*, BFS and UCS need the full state space and the results of all actions in advance.
- **Limited sensing:** The rover can sense only nearby cells, so the plan cannot be completed in advance.
- **Dynamic terrain:** Dust storms, rockfalls or shifting sand can change the terrain after a plan is made.
- **Communication delay:** Communication with Earth has a delay of several minutes (up to more than 20 minutes one way), so the rover must decide locally and quickly.
- **Exploration cost:** Some actions are costly or irreversible (getting stuck in a crater), so the agent must learn about the terrain step by step and act carefully.

ii) Challenges of Partial Observability and Dynamic Environments

Challenge	Explanation
Incomplete state information	The rover knows only the cells in its sensing range. It cannot tell what lies beyond, so it may enter a dead end.
Sensor noise and uncertainty	Rocks or craters may be missed or misclassified, which leads to unsafe decisions.
Dynamic terrain	Information already stored in the map can become outdated; obstacles may appear or disappear.
Exploration vs. exploitation	The rover must balance exploring unknown areas against moving toward the sample site efficiently.
Irreversible actions	A wrong move (falling into a crater, wheel stuck in sand) cannot be undone, so safety must have priority.
Resource limits	Battery, computation time and memory are limited, which restricts the complexity of the search.
Repeated visits and loops	Without memory, the rover may revisit the same cells again and again.

iii) How the Rover Builds and Updates its Internal Model

The rover maintains a model of the world and updates it after every action. The cycle is:

1. **Perceive:** Read the sensors (camera, LIDAR, proximity sensors) to detect nearby cells.

2. **Update the map:** Add the new cells to an occupancy grid or map with labels: free, obstacle (rock), hazard (crater), unknown. Update the confidence of each cell.
3. **Update the state memory:** Store the current position, visited cells, action results (for example, 'moving Up from cell X led to cell Y') and the cost estimates H(s) for each visited state.
4. **Plan:** Run a local search on the known map to choose the next action (for example, the cheapest and safest move toward the target or toward the nearest unexplored frontier).
5. **Act:** Execute one step, then return to the perceive step.

Learning rule for LRTA* (Learning Real-Time A*): After each move from state s, the rover updates its cost estimate:

$$H(s) = \min_{\text{over actions } a} [\, c(s, a, s') + H(s') \,]$$

The rover then moves to the neighbour with the lowest value. Estimates for visited states increase when they prove to be in dead ends, which pushes the rover away from them and prevents infinite loops.

iv) Comparison of Online Search with Uninformed and Informed Search

Feature	Uninformed (BFS, DFS, UCS)	Informed (Greedy, A*)	Online search (Online DFS, LRTA*, D* Lite)
Map requirement	Complete known map	Complete known map (plus heuristic)	Not required; learned during exploration
Planning style	Offline: plan, then act	Offline: plan, then act	Interleaved: plan, act, sense, update
Partial observability	Not handled	Not handled directly	Designed for it
Optimality	BFS and UCS optimal on the known map	A* optimal with an admissible h	Not optimal in general, but improves with experience (can match optimal if the map becomes known)
Memory and time	High (BFS) or risky (DFS)	Moderate to high	Low; only local information and the learned estimates
Safety in unknown terrain	Plans on assumptions, which may be unsafe	Same problem	Checks the terrain before moving and replans when hazards appear
Adaptability	Must restart the search after any change	Must restart or repair the search	Continuously adapts

v) Most Suitable Approach and Justification

Recommendation: An online search agent such as LRTA* or D* Lite, using a hazard-aware cost function, with a safety-first policy.

- **Handles uncertainty:** The agent never needs the full map. It works on the portion it has sensed and extends it with each move.
- **Adaptability:** The model and cost estimates update after each percept, and D* Lite repairs the plan efficiently when a new obstacle is seen.

- **Safety:** Hazards (craters, large rocks) get very high or infinite cost. The rover moves only into cells that the sensors have verified as safe. Unknown cells near hazards are treated cautiously, and the rover keeps a retreat path to previously visited safe cells.
- **Real-time performance:** Each decision uses a small local search, so it fits the limited computation and communication delay.
- **Completeness in a safely explorable space:** The learned H(s) values prevent revisiting dead ends, so the rover completes exploration instead of looping.

Question 4: University Exam Timetable as a Constraint Satisfaction Problem

A university designs an AI system to generate exam timetables. Constraints: no student has overlapping exams, a limited number of exam halls, some subjects must be scheduled before others, faculty availability, and special accommodations for some students. Complexity grows with the number of courses and students.

Task: Formulation as a CSP

A CSP is defined as (X, D, C): a set of variables X, their domains D, and a set of constraints C. A solution is a complete assignment of values to all variables that satisfies every constraint.

i) Variables, Domains and Constraints

Variables:

- Ti = time slot of the exam of course i (for every course i = 1 ... n).
- Ri = exam hall assigned to course i.
- Vi = invigilator or faculty member assigned to course i (optional).

Domains:

- D(Ti) = {1, 2, ..., S}: the available exam slots (for example, 3 slots per day × number of exam days).
- D(Ri) = {H1, H2, ..., Hm}: the available exam halls.
- D(Vi) = the set of faculty members eligible for course i.

Constraints:

No.	Constraint	Type	Formal form and explanation
1	No student has overlapping exams	Binary	For every pair of courses i, j that share at least one student: $Ti \neq Tj$. This forms a conflict graph, so the problem resembles graph colouring.
2	Limited number of exam halls	Binary / unary	If $Ti = Tj$ then $Ri \neq Rj$ (a hall hosts one exam at a time). Also capacity(Ri) ≥ enrolment(i) (unary).
3	Some subjects before others	Binary	If course i must precede course j: $Ti < Tj$.
4	Faculty availability	Unary / binary	$Ti \in$ Available(Vi). A faculty member cannot invigilate two exams in the same slot: if $Vi = Vj$ then $Ti \neq Tj$.
5	Special accommodations	Unary / binary	Students needing extra time or a separate room: $Ri \in$ AccessibleHalls, extra slot duration, and no back-to-back exams for such students: $\|Ti - Tj\| \geq 2$.

ii) Working of Backtracking Search in this Scenario

Backtracking search is a depth-first search that assigns one variable at a time and checks the constraints after each assignment. If a constraint is violated, the assignment is rejected, and if no value works the search returns to the previous variable (backtracks).

Worked example (simplified, only time slots): Courses C1 (Maths), C2 (Physics), C3 (Chemistry), C4 (Programming). Slots = {1, 2, 3}.

- Student conflicts (different slots required): C1–C2, C1–C3, C2–C3, C2–C4.
- Precedence: C3 must be before C4 (T3 < T4).
- Faculty availability: the C3 faculty are not available in slot 1.

Variable order: C1, C2, C3, C4. Value order: 1, 2, 3.

Step	Assignment attempted	Constraint check	Result
1	C1 = 1	No conflicts yet	Accept
2	C2 = 1	Conflict C1–C2 (same slot)	Reject
3	C2 = 2	C1–C2 satisfied	Accept
4	C3 = 1	Faculty unavailable; conflict with C1	Reject
5	C3 = 2	Conflict C2–C3	Reject
6	C3 = 3	C1–C3, C2–C3, faculty satisfied	Accept
7	C4 = 1, 2, 3	Precedence T3 < T4 requires T4 > 3; no value works (also C2–C4 for slot 2)	Dead end: backtrack
8	Back to C3	No more values (all tried)	Backtrack to C2
9	C2 = 3	C1–C2 satisfied	Accept
10	C3 = 1, 2	C3 = 1: faculty unavailable. C3 = 2: valid with C1 = 1 and C2 = 3	C3 = 2 accepted
11	C4 = 1, 2, 3	1 and 2 violate T3 < T4; 3 conflicts with C2 (slot 3)	Dead end: backtrack
12	C3 = 3	Conflict with C2 = 3	Reject; C3 exhausted, so backtrack to C2, then C1
13	C1 = 2	With C1 = 2, the branches C2 = 1 and C2 = 3 both fail (as in steps 4–12: C3 has no workable value, or C4 has none)	Backtrack to C1
14	C1 = 3, C2 = 1, C3 = 2, C4 = 3	C1–C2, C1–C3, C2–C3, C2–C4 satisfied; faculty available (slot 2); T3 = 2 < T4 = 3	Solution found

Final timetable: C2 (Physics) in slot 1, C3 (Chemistry) in slot 2, C1 (Maths) and C4 (Programming) in slot 3. The number of rejected assignments shows how plain backtracking wastes effort on partial assignments that were doomed earlier.

iii) Constraint Propagation (Forward Checking) and Improved Efficiency

Forward checking: whenever a variable is assigned, remove the inconsistent values from the domains of its unassigned neighbours. If any domain becomes empty, backtrack immediately.

Applied to the same example:

After assignment	D(C1)	D(C2)	D(C3)	D(C4)	Remark
Initial (with C3 unavailable in slot 1)	{1,2,3}	{1,2,3}	{2,3}	{1,2,3}	—

After assignment	D(C1)	D(C2)	D(C3)	D(C4)	Remark
C1 = 1	{1}	{2,3}	{2,3}	{1,2,3}	1 removed from C2 and C3 (conflicts)
C2 = 2	{1}	{2}	{3}	{1,3}	2 removed from C3 (conflict) and C4 (conflict C2–C4)
C3 = 3	{1}	{2}	{3}	{ } (empty)	Precedence T3 < T4 leaves no value for C4: failure detected immediately

Forward checking detects the dead end as soon as C3 = 3 is assigned, without trying C4 = 1, 2, 3 one by one. Further improvements:

- **Minimum Remaining Values (MRV):** Selects the variable with the fewest remaining values first (for example, C3, which has only two values), so failures are found early.
- **Degree heuristic:** Chooses the variable involved in the most constraints (the course with most shared students) first.
- **Least Constraining Value:** Tries first the value that removes the fewest options from neighbours.
- **Arc consistency:** Maintains arc consistency (AC-3), which propagates the reduction of domains further than forward checking and catches more failures early.

iv) Real-World Challenges and Best Strategy for Scalability

Challenge	Explanation
Large size	Thousands of students and hundreds of courses give a very large and densely connected conflict graph.
Over-constrained problem	Often no timetable satisfies all constraints. Some constraints (spacing, student preferences) must be treated as soft constraints with penalties.
Last-minute changes	Faculty absence, hall maintenance or late registrations require quick repair of the timetable instead of a complete rebuild.
Hall capacity (bin packing)	Combining several exams in one hall in the same slot adds a capacity-packing problem.
Fairness and quality	Students should not have many exams on consecutive days, and gaps should be reasonable.

Recommended strategy: Backtracking search enhanced with MRV, the degree heuristic, forward checking or AC-3 and good value ordering, to construct a feasible timetable satisfying all hard constraints. Then use local search (min-conflicts or simulated annealing) to improve soft constraints and to repair the timetable when changes occur.

Justification: Heuristic-guided backtracking with propagation prunes large parts of the search space and guarantees that hard constraints are satisfied. Local search scales well to very large instances and handles dynamic changes by repairing only the affected exams. Decomposing the problem (by department or by day) further improves scalability.

Question 5: Strategic Game AI (Minimax and Alpha-Beta Pruning)

A gaming company develops an AI opponent for a real-time strategy game. The AI must compete against human players, decide within strict time limits, evaluate complex game states with many possible moves, and the decision tree is extremely large, making computation expensive.

i) How the Minimax Algorithm Models the Problem

Minimax models a two-player, zero-sum, adversarial game. The AI is the MAX player (tries to maximize the utility) and the human is the MIN player (tries to minimize the AI's utility).

Game element	In the strategy game
State s	Positions of units, resources, buildings, map control and health of both players
Actions(s)	Legal moves: build, move or attack with units, gather resources, research
Transition model	Result of a move on the game state (the real-time game is discretized into time steps or turns)
Terminal test	A base is destroyed, one side is eliminated, or the time limit is reached
Utility	+1 (win), −1 (loss), 0 (draw); in practice an evaluation score at the depth limit

Minimax value:

MINIMAX(s) = UTILITY(s) if s is terminal; max over actions of MINIMAX(RESULT(s, a)) if it is MAX's turn; min over actions of MINIMAX(RESULT(s, a)) if it is MIN's turn.

The AI assumes the human plays optimally (picks moves that are worst for the AI), so it chooses the move with the best guaranteed outcome. A real-time game is not strictly turn-based and has hidden information, so it is approximated by discretizing time and assuming alternating moves.

ii) Computational Challenges of Large Game Trees

- **Exponential growth:** The number of nodes is about b^d, where b is the branching factor and d the depth. With b = 30 and d = 10, there are about 30^10 ≈ 5.9 × 10^14 nodes, which is impossible to search in a few milliseconds.
- **Real-time limit:** Strict decision time limits mean the full tree cannot be explored, so the search must stop at a depth limit.
- **Expensive evaluation:** Evaluating complex game states (many units, resources, map control) takes time at each node.
- **Horizon effect:** Stopping at a fixed depth can hide a dangerous event just beyond the horizon (for example, an attack that is coming).
- **Repeated states:** The same game state can be reached by different move orders, so the algorithm repeats work.
- **Memory:** Storing a large tree uses much memory; depth-first search keeps it manageable, but the time cost remains.

iii) How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta pruning computes the same result as Minimax but skips branches that cannot influence the final decision. It keeps two values:

- Alpha (α) = the best (highest) value found so far for MAX along the current path.
- Beta (β) = the best (lowest) value found so far for MIN along the current path.

Pruning rule: if at any node α ≥ β, stop exploring the remaining children of that node, because the opponent will never allow the game to reach it.

Worked example: MAX root with three MIN nodes A, B, C. Leaf values: A = (3, 12, 8), B = (2, 4, 6), C = (14, 5, 2).

Step	Node evaluated	α	β	Reasoning
1	A: leaf 3	−∞	3	MIN node A so far has value ≤ 3
2	A: leaves 12 and 8	−∞	3	MIN keeps the minimum, so A = 3. Root α becomes 3
3	B: leaf 2	3	2	β = 2 ≤ α = 3: B can be at most 2, which is worse for MAX than A = 3
4	B: leaves 4 and 6	3	2	PRUNED: not evaluated, because B cannot beat 3
5	C: leaf 14	3	14	No pruning yet (β > α)
6	C: leaf 5	3	5	Still above α
7	C: leaf 2	3	2	C = 2 ≤ 3, so C is rejected
8	Root	3	—	MAX chooses A with value 3 (same as Minimax)

Detailed reasoning:

- The pruned leaves (4 and 6) could not change the result, because MIN at B already has a reply (value 2) that is worse for MAX than the guaranteed 3 from A. So the outcome is identical but with fewer evaluations.
- Pruning depends on move ordering. In the best case (strongest moves first) the complexity falls from O(b^d) to O(b^(d/2)), so the same time allows the search to go twice as deep. For b = 30 and d = 10, the cost drops from about 5.9 × 10^14 to about 30^5 ≈ 2.4 × 10^7 nodes.
- In the worst case (poor ordering) no pruning occurs and the cost is the same as Minimax.
- Good ordering methods: try captures or attacks first, killer moves, history heuristic and the best move from the previous iteration.

iv) Design of an Evaluation Function

Because the tree cannot be searched to the end, the search stops at a depth limit and estimates the quality of the state with an evaluation function, as a weighted linear sum of features:

$$\mathbf{Eval(s) = w1 \cdot f1 + w2 \cdot f2 + w3 \cdot f3 + w4 \cdot f4 + w5 \cdot f5 + w6 \cdot f6}$$

Feature	Definition (AI minus opponent)	Justification
f1: Army strength	Total attack and health value of the units	Strong armies win battles and protect the base
f2: Resources and economy	Stored resources and income rate (workers, mines)	Economy decides how many units can be produced later
f3: Map and territory control	Number of controlled zones, bases or key points	Control gives vision, resources and positional advantage
f4: Technology / upgrades	Level of researched upgrades and unlocked unit types	Better technology gives a long-term advantage
f5: Base safety	Health of main buildings and defence level	Losing the base means losing the game
f6: Threat and mobility	Enemy units near the AI's base and the AI's freedom to move	Detects immediate danger or opportunities

- **Terminal states:** Winning and losing states receive a very large positive or negative value (for example, ±1000).
- **Normalization and weights:** Each feature is scaled to the same range (for example, −1 to +1) so that no single feature dominates. Example weights: w1 = 0.30, w2 = 0.20, w3 = 0.15, w4 = 0.10, w5 = 0.15, w6 = 0.10 (weights are tuned by self-play or machine learning).
- **Desirable properties:** Fast to compute, strongly correlated with the chance of winning, and uses only observable information.

v) Strategy for Balancing Optimality and Real-Time Performance

Recommended strategy: Iterative deepening Alpha-Beta search with a time limit, supported by move ordering, a transposition table and a good evaluation function.

1. **Iterative deepening with a time limit:** Search to depth 1, then 2, 3, and so on. When time runs out, return the best move from the last completed depth. This gives an anytime algorithm that always has an answer.
2. **Move ordering:** Order moves using the previous iteration's best move, killer moves and the history heuristic so that Alpha-Beta prunes as much as possible.
3. **Transposition table:** Store evaluated states in a hash table so that repeated positions are not searched again.
4. **Quiescence search / selective deepening:** Extend the search in volatile positions (active combat) before evaluating, to reduce the horizon effect.
5. **Forward pruning and abstraction:** Prune unpromising moves in a limited-width (beam) search and abstract the game (group units into squads, coarse map areas) to cut the branching factor.
6. **Hybrid options:** If the branching factor is still too large, combine with Monte-Carlo Tree Search to sample promising lines.

Justification: This approach gives the best possible move within the available time. It keeps the optimality of Minimax for the searched depth, while the pruning and time management guarantee a decision within the strict real-time limit.